

PayFlow - Complete Project Documentation

Table of Contents

- 1. [Project Overview](#)
- 2. [System Architecture](#)
- 3. [Technologies Stack](#)
- 4. [Project Structure](#)
- 5. [Installation & Setup](#)
- 6. [Configuration](#)
- 7. [Database Schema](#)
- 8. [API Documentation](#)
- 9. [Features Documentation](#)
- 10. [Frontend Architecture](#)
- 11. [Backend Architecture](#)
- 12. [Security & Authentication](#)
- 13. [Deployment](#)
- 14. [Testing](#)
- 15. [Contributing Guidelines](#)
- 16. [Troubleshooting](#)

Project Overview

What is PayFlow?

PayFlow is a lightweight yet powerful **Customer Relationship Management (CRM)** and **Billing Platform** designed to help startups, freelancers, and small businesses eliminate operational chaos. With integrated AI insights and seamless payment handling, PayFlow makes business management smarter and simpler.

Project Team

- **Bassem Hazem**
- **Seraj Eldeen**
- **Amira Ahmed**
- **Alaa Nabil**
- **Marwa Hassan**

Key Features

- **AI Assistant** — Smart assistant that helps automate business decisions
- **Smart Invoices** — Automatically generate and send invoices to clients
- **Smart Products** — Auto-generate product details and pricing
- **Smart Customers** — Manage and auto-create customer profiles
- **Smart Billing** — Handle automated billing with Stripe integration
- **Smart Payments** — Track and confirm payments in real-time
- **Smart Reports** — Auto-generate financial and activity reports

- **Arabic Support** — Multi-language support for Arabic and English
- **Admin Panel** — Manage users, see user activity, and permissions
- **User Dashboard** — Personalized dashboard for quick insights
- **Dark Mode** — A smooth and elegant dark interface option
- **Stripe Integration** — Secure and seamless online payment processing

System Architecture

Architecture Pattern

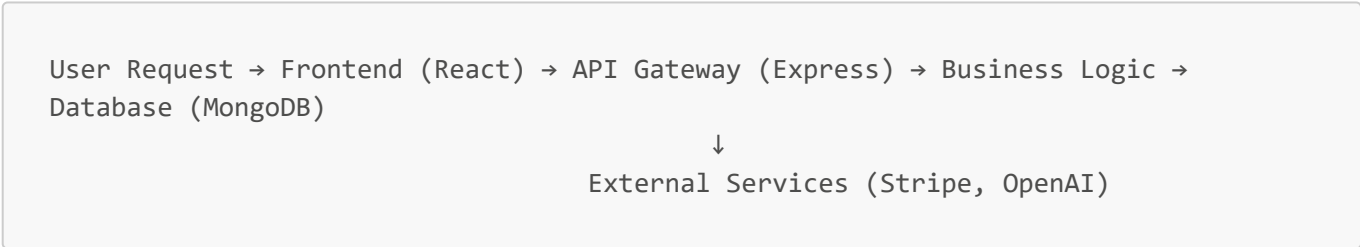
PayFlow follows a **3-tier architecture** pattern:

1. **Presentation Layer (Frontend)** - React.js with Vite
2. **Business Logic Layer (Backend)** - Node.js with Express.js
3. **Data Layer** - MongoDB with Mongoose ODM

Architecture Principles

- **RESTful API Design** - Standardized HTTP methods and status codes
- **Token-based Authentication** - JWT (JSON Web Tokens) for secure user sessions
- **Modular & Scalable Structure** - Feature-based folder organization
- **AI-based Automation Layer** - OpenAI integration for intelligent features
- **Microservices-Ready** - Modular design allows easy service separation

System Flow



Technologies Stack

Backend Technologies

Technology	Version	Purpose
Node.js	-	Runtime environment
Express.js	^5.1.0	Web framework
MongoDB	^6.18.0	Database
Mongoose	^8.17.2	ODM for MongoDB
JWT	^9.0.2	Authentication
Bcrypt.js	^3.0.2	Password encryption

Technology	Version	Purpose
Stripe	^18.5.0	Payment processing
OpenAI	^6.3.0	AI assistance
Passport	^0.7.0	OAuth authentication
Redis	^5.8.1	Caching
Multer	^2.0.2	File uploads
PDFKit	^0.17.2	PDF generation
ExcelJS	^4.4.0	Excel generation
Node-Cron	^4.2.1	Scheduled tasks
Helmet	^8.1.0	Security middleware
Morgan	^1.10.1	Logging
CORS	^2.8.5	Cross-origin resource sharing
Express Rate Limit	^8.0.1	Rate limiting

Frontend Technologies

Technology	Version	Purpose
React.js	^18.3.1	UI library
Vite	-	Build tool
Redux Toolkit	^2.9.0	State management
React Router	^7.8.1	Routing
TanStack Query	^5.85.3	Data fetching
Axios	^1.12.2	HTTP client
Tailwind CSS	^3.4.18	Styling
Framer Motion	^12.23.24	Animations
i18next	^25.6.0	Internationalization
React Hook Form	^7.64.0	Form management
Recharts	^3.2.1	Data visualization
React Hot Toast	^2.6.0	Notifications
Lucide React	^0.545.0	Icons
JWT Decode	^4.0.0	JWT decoding
jsPDF	^3.0.3	PDF generation

Technology	Version	Purpose
html2canvas	^1.4.1	Screenshot capture

DevOps & Deployment

- **Vercel** - Hosting platform
- **Git** - Version control
- **Nodemon** - Development auto-reload
- **Concurrently** - Run multiple scripts

Project Structure

PayFlow/

— backend/

Backend application

— src/

Configuration files

— config/

MongoDB connection

— db.js

Passport OAuth config

— passport.js

Redis configuration

— redis.js

Stripe configuration

— stripe.js

Database models

— models/

User schema

— user.model.js

Company schema

— company.model.js

Customer schema

— customer.model.js

Product schema

— product.model.js

Invoice schema

— invoice.model.js

Report schema

— report.model.js

pricing_billing.model.js

Feature modules

— modules/

Authentication

— auth/

— auth.controller.js

— auth.routes.js

Company management

— company/

Customer management

— customer/

Product management

— product/

Invoice management

— invoice/

Reports generation

— report/

Billing & payments

— pricing_billing/

AI features

— AiAssistance/

Utility functions

— utils/

JWT utilities

— jwt.js

Usage tracking

— trackUsage.js

Feature utilities

— features/

Error handlers

— handlers/

Custom middlewares

— middlewares/

└─ services/	# Shared services
└─ seed/	# Database seeding
└─┬─ seed.js	
└─ index.js	# Application entry point
└─ uploads/	# Uploaded files
└─┬─ company-logos/	
└─ package.json	# Backend dependencies
└─ vercel.json	# Vercel configuration
└─ frontend/	# Frontend application
└─┬─ src/	
└─┬─┬─ api/	# API integration
└─┬─┬─┬─ axios.js	# Axios configuration
└─┬─┬─┬─ ai.js	# AI API calls
└─┬─┬─┬─ users.js	# User API calls
└─┬─┬─┬─ company.js	# Company API calls
└─┬─┬─┬─ customer.js	# Customer API calls
└─┬─┬─┬─ product.js	# Product API calls
└─┬─┬─┬─ invoice.js	# Invoice API calls
└─┬─┬─┬─ report.js	# Report API calls
└─┬─┬─┬─ pricing_billing.js	# Billing API calls
└─┬─┬─ app/	# App-level configs
└─┬─┬─┬─ QueryProvider.jsx	# React Query setup
└─┬─┬─ assets/	# Static assets
└─┬─┬─┬─ fonts/	# Font files
└─┬─┬─┬─ products/	# Product images
└─┬─┬─┬─ dashboard.png	# Dashboard preview
└─┬─┬─┬─ play-icon.png	# Play button icon
└─┬─┬─┬─ dashboard-icon.png	# Dashboard icon
└─┬─┬─┬─ ai-icon.png	# AI assistant icon
└─┬─┬─┬─ reports-icon.png	# Reports icon
└─┬─┬─┬─ products-icon.png	# Products icon
└─┬─┬─┬─ database-icon.png	# Database icon
└─┬─┬─┬─ invoice-icon-b.png	# Invoice icon
└─┬─┬─┬─ ...	# Other icons/images
└─┬─┬─ components/	# React components
└─┬─┬─┬─ ui/	# Reusable UI components
└─┬─┬─┬─┬─ Button.jsx	# Reusable button
└─┬─┬─┬─┬─ FeatureCard.jsx	# Feature card component
└─┬─┬─┬─┬─ FocusedCards.jsx	# Focused cards component
└─┬─┬─┬─┬─ StepCard.jsx	# Step card component
└─┬─┬─┬─┬─ HomeNavbar.jsx	# Landing page navbar
└─┬─┬─┬─┬─ Footer.jsx	# Footer component
└─┬─┬─┬─┬─ TestimonialSlider.jsx	# Testimonial slider
└─┬─┬─┬─┬─ FloatingWhatsApp.jsx	# WhatsApp widget
└─┬─┬─┬─┬─ Table.jsx	# Table component
└─┬─┬─┬─┬─ ProgressBar.jsx	# Progress bar

└─ SystemManagement.jsx	# System management UI
└─ UnregisterHome.jsx	# Landing page (Home)
└─ Dashboard.jsx	# Main dashboard
└─ Login.jsx	# Login page
└─ SignUp.jsx	# Signup page
└─ Products.jsx	# Products page
└─ AddProduct.jsx	# Add product page
└─ EditProduct.jsx	# Edit product page
└─ ViewProduct.jsx	# View product details
└─ Customers.jsx	# Customers page
└─ AddCustomer.jsx	# Add customer page
└─ EditCustomer.jsx	# Edit customer page
└─ ViewCustomer.jsx	# View customer details
└─ Invoices.jsx	# Invoices page
└─ AddInvoice.jsx	# Add/Edit invoice page
└─ InvoicePreviewPage.jsx	# Invoice preview/print
└─ Reports.jsx	# Reports page
└─ Pricing_billing.jsx	# Billing page
└─ AiAssistant.jsx	# AI assistant
└─ ChatComponent.jsx	# Chat interface
└─ AdminDashboard.jsx	# Admin panel
└─ AdminComponent.jsx	# Admin layout wrapper
└─ UserComponent.jsx	# User layout wrapper
└─ Settings.jsx	# Settings page
└─ PrivateRoute.jsx	# Protected route wrapper
└─ GoogleCallback.jsx	# OAuth callback handler
└─ ErrorPage.jsx	# 404/Error page
└─ Loader.jsx	# Loading spinner
└─ Theme.jsx	# Theme switcher
└─ Topbar.jsx	# Top navigation bar
└─ MonthSelector.jsx	# Month selector component
└─ WobbleEffect.jsx	# Wobble animation effect
└─ features/	# Redux features
└─ └─ admin/	# Admin state management
└─ └─ company/	# Company state
└─ └─ customer/	# Customer state
└─ └─ invoice/	# Invoice state
└─ └─ product/	# Product state
└─ └─ report/	# Report state
└─ └─ user/	# User state
└─ └─ pricing_billing/	# Billing state
└─ lang/	# Translations
└─ └─ en.json	# English translations
└─ └─ ar.json	# Arabic translations
└─ lib/	# Library configurations
└─ └─ i18n.js	# i18n setup
└─ └─ utils.js	# Utility functions
└─ utils/	# Utilities
└─ └─ constants.js	# App constants

├── LanguageContext.jsx	# Language context
├── usePreferences.js	# Custom hooks
├── App.jsx	# Root component
├── main.jsx	# Entry point
├── index.css	# Global styles
├── public/	# Public assets
├── logo.png	# PayFlow logo
├── components.json	# shadcn/ui config
├── tailwind.config.js	# Tailwind config
├── postcss.config.js	# PostCSS config
├── vite.config.js	# Vite config
├── eslint.config.js	# ESLint config
├── package.json	# Frontend dependencies
├── package.json	# Root package.json
├── README.md	# Project README

Installation & Setup

Prerequisites

Before installing PayFlow, ensure you have:

- **Node.js** (v18 or higher)
- **npm** or **yarn**
- **MongoDB** (local or cloud instance)
- **Git**
- **Stripe Account** (for payment processing)
- **OpenAI API Key** (for AI features)
- **Google OAuth Credentials** (optional, for Google login)

Installation Steps

1. Clone the Repository

```
git clone https://github.com/SerajEldeen/PayFlow.git
cd PayFlow
```

2. Install Dependencies

Root Dependencies:

```
npm install
```

Backend Dependencies:

```
cd backend
npm install
cd ..
```

Frontend Dependencies:

```
cd frontend
npm install
cd ..
```

3. Environment Configuration

Create `.env` file in the `backend` directory:

```
# Server Configuration
PORT=3000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173

# Database Configuration
MONGO_URL=mongodb://localhost:27017
MONGO_DB=payflow

# JWT Configuration
JWT_SECRET=your_jwt_secret_key_here
JWT_EXPIRES_IN=7d

# Stripe Configuration
STRIPE_SECRET_KEY=sk_test_your_stripe_secret_key
STRIPE_PUBLISHABLE_KEY=pk_test_your_stripe_publishable_key
STRIPE_WEBHOOK_SECRET=whsec_your_webhook_secret

# OpenAI Configuration
OPENAI_API_KEY=sk-your_openai_api_key

# Google OAuth Configuration (Optional)
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_CALLBACK_URL=http://localhost:5000/api/auth/google/callback

# Redis Configuration (Optional)
REDIS_URL=redis://localhost:6379
UPSTASH_REDIS_REST_URL=your_upstash_redis_url
UPSTASH_REDIS_REST_TOKEN=your_upstash_token
```



```
# Email Configuration (Optional)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASSWORD=your_email_password
```

Create `.env` file in the `frontend` directory:

```
VITE_API_URL=http://localhost:3000/api
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_your_stripe_publishable_key
```

4. Database Setup

Seed the database with initial data:

```
cd backend
npm run seed
```

To delete all data and reseed:

```
npm run delete
```

5. Run the Application

Development Mode (runs both frontend and backend):

```
# From root directory
npm run dev
```

Or run separately:

```
# Terminal 1 - Backend
cd backend
npm run dev

# Terminal 2 - Frontend
cd frontend
npm run dev
```

6. Access the Application

- **Frontend:** <http://localhost:5173>
- **Backend API:** <http://localhost:3000>
- **API Documentation:** <http://localhost:3000/api>

Default Credentials (After Seeding)

Admin User:
Email: `admin@payflow.com`
Password: `admin123`

Regular User:
Email: `user@payflow.com`
Password: `user123`

Configuration

Backend Configuration

Database Configuration (`backend/src/config/db.js`)

```
import mongoose from "mongoose";

export const connectDB = () => {
  mongoose
    .connect(process.env.MONGO_URL, {
      dbName: process.env.MONGO_DB,
    })
    .then(() => console.log(`✅ MongoDB Connected to ${process.env.MONGO_DB}`))
    .catch((err) => console.log(err));
};
```

Configuration Details:

- Uses Mongoose ODM for MongoDB
- Database URL from `MONGO_URL` environment variable
- Database name from `MONGO_DB` environment variable
- Supports both local and cloud MongoDB instances (MongoDB Atlas)

Stripe Configuration (`backend/src/config/stripe.js`)

Configure Stripe for payment processing with products, prices, and webhook handling.

Passport Configuration (`backend/src/config/passport.js`)

Configure Google OAuth 2.0 authentication strategy.

Redis Configuration (backend/src/config/redis.js)

Configure Redis for caching and session management.

Frontend Configuration

Vite Configuration (frontend/vite.config.js)

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/auth': {
        target: 'http://localhost:3000',
        changeOrigin: true,
        secure: false,
      },
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true,
        secure: false,
      },
    },
  },
})
```

Tailwind Configuration (frontend/tailwind.config.js)

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["../src/**/*.{js,jsx,ts,tsx,html}"],
  darkMode: "class",
  theme: {
    extend: {
      colors: {
        primary: "#579BB1",
        secondary: "#E1D7C6",
        accent: "#ECE8DD",
        background: "#F8F4EA",
        primary_dark: "#579BB1",
        secondary_dark: "#2E535E",
        accent_dark: "#273f4e",
        background_dark: "#142129",
      },
    },
    fontFamily: {
      quicksand: ["Quicksand", "sans-serif"],
    },
  },
}
```

```
        robotoCondensed: ["Roboto Condensed", "sans-serif"],
      },
    },
  },
  plugins: [],
};
```

i18n Configuration (frontend/src/lib/i18n.js)

Configure internationalization for Arabic and English languages.

Database Schema

User Model

```
{
  firstName: String (required),
  lastName: String (required),
  email: String (required, unique),
  password: String (required, hashed),
  role: String (enum: ['user', 'admin']),
  company: ObjectId (ref: 'Company'),
  googleId: String,
  profileImage: String,
  isActive: Boolean,
  lastLogin: Date,
  createdAt: Date,
  updatedAt: Date
}
```

Company Model

```
{
  name: String (required),
  owner: ObjectId (ref: 'User'),
  logo: String,
  address: String,
  phone: String,
  email: String,
  website: String,
  taxId: String,
  industry: String,
  employees: [ObjectId] (ref: 'User'),
  createdAt: Date,
  updatedAt: Date
}
```

Customer Model

```
{
  name: String (required),
  email: String (required),
  phone: String,
  address: String,
  company: ObjectId (ref: 'Company'),
  user: ObjectId (ref: 'User'),
  notes: String,
  status: String (enum: ['active', 'inactive']),
  totalInvoices: Number,
  totalRevenue: Number,
  createdAt: Date,
  updatedAt: Date
}
```

Product Model

```
{
  name: String (required),
  description: String,
  price: Number (required),
  category: String,
  sku: String,
  stock: Number,
  image: String,
  company: ObjectId (ref: 'Company'),
  user: ObjectId (ref: 'User'),
  isActive: Boolean,
  createdAt: Date,
  updatedAt: Date
}
```

Invoice Model

```
{
  invoiceNumber: String (required, unique),
  customer: ObjectId (ref: 'Customer'),
  company: ObjectId (ref: 'Company'),
  user: ObjectId (ref: 'User'),
  items: [{
    product: ObjectId (ref: 'Product'),
    quantity: Number,
    price: Number,
    total: Number
  }],
  subtotal: Number,
```

```
tax: Number,
discount: Number,
total: Number,
status: String (enum: ['draft', 'sent', 'paid', 'overdue', 'cancelled']),
issueDate: Date,
dueDate: Date,
paidDate: Date,
notes: String,
paymentMethod: String,
createdAt: Date,
updatedAt: Date
}
```

Report Model

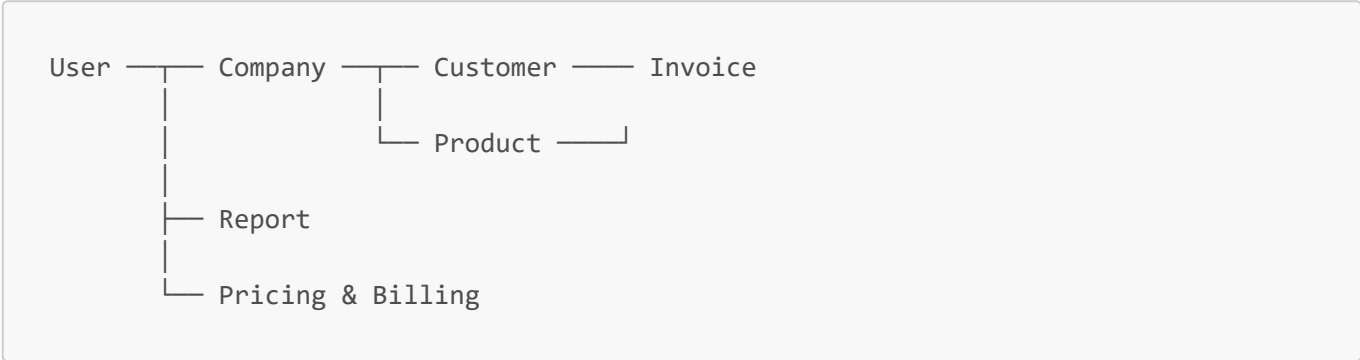
```
{
  title: String (required),
  type: String (enum: ['sales', 'revenue', 'customers', 'products']),
  company: ObjectId (ref: 'Company'),
  user: ObjectId (ref: 'User'),
  period: {
    startDate: Date,
    endDate: Date
  },
  data: Object,
  generatedAt: Date,
  format: String (enum: ['pdf', 'excel', 'csv']),
  fileUrl: String
}
```

Pricing & Billing Model

```
{
  user: ObjectId (ref: 'User'),
  plan: String (enum: ['free', 'basic', 'premium', 'enterprise']),
  status: String (enum: ['active', 'cancelled', 'past_due']),
  stripeCustomerId: String,
  stripeSubscriptionId: String,
  currentPeriodStart: Date,
  currentPeriodEnd: Date,
  cancelAtPeriodEnd: Boolean,
  usage: {
    invoices: Number,
    customers: Number,
    products: Number,
    aiRequests: Number
  },
  limits: {
    invoices: Number,
```

```
    customers: Number,
    products: Number,
    aiRequests: Number
  },
  createdAt: Date,
  updatedAt: Date
}
```

Entity Relationship



API Documentation

Authentication Endpoints

Register User

```
POST /api/auth/register
Content-Type: application/json

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "password": "password123"
}

Response: 201 Created
{
  "success": true,
  "message": "User registered successfully",
  "user": { ... },
  "token": "jwt_token_here"
}
```

Login User

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "email": "john@example.com",
  "password": "password123"
}
```

Response: 200 OK

```
{
  "success": true,
  "user": { ... },
  "token": "jwt_token_here"
}
```

Google OAuth Login

```
GET /api/auth/google
```

Logout User

```
POST /api/auth/logout
Authorization: Bearer {token}
```

Response: 200 OK

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

Get Current User

```
GET /api/auth/me
Authorization: Bearer {token}
```

Response: 200 OK

```
{
  "success": true,
  "user": { ... }
}
```

Company Endpoints

Create Company

```
POST /api/company
Authorization: Bearer {token}
Content-Type: multipart/form-data
```

```
{
  "name": "Company Name",
  "address": "123 Street",
  "phone": "+1234567890",
  "email": "company@example.com",
  "logo": File
}
```

Response: 201 Created

Get Company

```
GET /api/company/:id
Authorization: Bearer {token}
```

Response: 200 OK

Update Company

```
PUT /api/company/:id
Authorization: Bearer {token}
```

Response: 200 OK

Delete Company

```
DELETE /api/company/:id
Authorization: Bearer {token}
```

Response: 204 No Content

Customer Endpoints

Create Customer

```
POST /api/customer
Authorization: Bearer {token}
Content-Type: application/json

{
  "name": "Customer Name",
  "email": "customer@example.com",
  "phone": "+1234567890",
  "address": "Customer Address"
}

Response: 201 Created
```

Get All Customers

```
GET /api/customer
Authorization: Bearer {token}

Query Parameters:
- page (default: 1)
- limit (default: 10)
- search
- status

Response: 200 OK
{
  "success": true,
  "customers": [...],
  "pagination": { ... }
}
```

Get Customer by ID

```
GET /api/customer/:id
Authorization: Bearer {token}

Response: 200 OK
```

Update Customer

```
PUT /api/customer/:id
Authorization: Bearer {token}

Response: 200 OK
```

Delete Customer

```
DELETE /api/customer/:id
Authorization: Bearer {token}
```

Response: 204 No Content

Product Endpoints

Create Product

```
POST /api/product
Authorization: Bearer {token}
Content-Type: multipart/form-data
```

```
{
  "name": "Product Name",
  "description": "Product Description",
  "price": 99.99,
  "category": "Category",
  "stock": 100,
  "image": File
}
```

Response: 201 Created

Get All Products

```
GET /api/product
Authorization: Bearer {token}
```

Query Parameters:

- page
- limit
- search
- category
- minPrice
- maxPrice

Response: 200 OK

Get Product by ID

```
GET /api/product/:id
Authorization: Bearer {token}

Response: 200 OK
```

Update Product

```
PUT /api/product/:id
Authorization: Bearer {token}

Response: 200 OK
```

Delete Product

```
DELETE /api/product/:id
Authorization: Bearer {token}

Response: 204 No Content
```

Invoice Endpoints

Create Invoice

```
POST /api/invoice
Authorization: Bearer {token}
Content-Type: application/json

{
  "customer": "customer_id",
  "items": [
    {
      "product": "product_id",
      "quantity": 2,
      "price": 99.99
    }
  ],
  "tax": 10,
  "discount": 5,
  "dueDate": "2025-12-31",
  "notes": "Invoice notes"
}

Response: 201 Created
```

Get All Invoices

```
GET /api/invoice
Authorization: Bearer {token}
```

Query Parameters:

- page
- limit
- status
- customer
- startDate
- endDate

Response: 200 OK

Get Invoice by ID

```
GET /api/invoice/:id
Authorization: Bearer {token}
```

Response: 200 OK

Update Invoice

```
PUT /api/invoice/:id
Authorization: Bearer {token}
```

Response: 200 OK

Delete Invoice

```
DELETE /api/invoice/:id
Authorization: Bearer {token}
```

Response: 204 No Content

Generate Invoice PDF

```
GET /api/invoice/:id/pdf
Authorization: Bearer {token}
```

Response: PDF File

Send Invoice Email

```
POST /api/invoice/:id/send
Authorization: Bearer {token}
```

Response: 200 OK

Report Endpoints

Generate Report

```
POST /api/reports
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "type": "sales",
  "period": {
    "startDate": "2025-01-01",
    "endDate": "2025-12-31"
  },
  "format": "pdf"
}
```

Response: 201 Created

Get All Reports

```
GET /api/reports
Authorization: Bearer {token}
```

Response: 200 OK

Download Report

```
GET /api/reports/:id/download
Authorization: Bearer {token}
```

Response: File Download

Pricing & Billing Endpoints

Get Current Plan

```
GET /api/pricing_billing/plan
Authorization: Bearer {token}
```

Response: 200 OK

Create Checkout Session

```
POST /api/pricing_billing/create-checkout-session
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "priceId": "price_id_from_stripe"
}
```

Response: 200 OK

```
{
  "sessionId": "checkout_session_id"
}
```

Cancel Subscription

```
POST /api/pricing_billing/cancel
Authorization: Bearer {token}
```

Response: 200 OK

Get Usage Statistics

```
GET /api/pricing_billing/usage
Authorization: Bearer {token}
```

Response: 200 OK

AI Assistance Endpoints

Chat with AI

```
POST /api/ai/chat
Authorization: Bearer {token}
```

```
Content-Type: application/json

{
  "message": "Help me create an invoice",
  "context": { ... }
}

Response: 200 OK
{
  "response": "AI response here",
  "suggestions": [ ... ]
}
```

Generate Product Description

```
POST /api/ai/generate-product
Authorization: Bearer {token}
Content-Type: application/json

{
  "productName": "Product Name",
  "category": "Category"
}

Response: 200 OK
```

Generate Customer Profile

```
POST /api/ai/generate-customer
Authorization: Bearer {token}

Response: 200 OK
```

Features Documentation

1. Authentication System

Technologies: JWT, Bcrypt, Passport.js, Google OAuth

Features:

- Email/Password registration and login
- Google OAuth 2.0 integration
- JWT token-based authentication
- Secure password hashing with bcrypt
- Token refresh mechanism

- Cookie-based token storage
- Protected routes

Implementation:

- Tokens expire after 7 days (configurable)
- Passwords hashed with bcrypt (salt rounds: 10)
- OAuth callback handling for Google sign-in
- Middleware for route protection

2. Company Management

Features:

- Create and manage company profiles
- Upload company logos
- Manage company information (address, phone, email, tax ID)
- Track company employees
- Company-specific data isolation

3. Customer Management

Features:

- Add, edit, delete customers
- Customer profile management
- Track customer invoices and revenue
- Customer activity tracking
- Search and filter customers
- Customer status management (active/inactive)
- AI-powered customer profile generation

4. Product Management

Features:

- Add, edit, delete products
- Upload product images
- Manage product inventory
- Product categorization
- SKU management
- Price management
- Stock tracking
- AI-powered product description generation

5. Invoice Management

Features:

- Create professional invoices
- Multiple invoice items per invoice

- Automatic invoice numbering
- Tax and discount calculations
- Invoice status tracking (draft, sent, paid, overdue, cancelled)
- PDF invoice generation
- Email invoice delivery
- Invoice preview
- Print invoices
- Payment tracking
- Due date management

6. Reports & Analytics

Features:

- Sales reports
- Revenue reports
- Customer reports
- Product reports
- Custom date range selection
- Export to PDF, Excel, CSV formats
- Visual charts and graphs (using Recharts)
- Dashboard analytics

Report Types:

- **Sales Report:** Total sales, sales by period, top products
- **Revenue Report:** Revenue trends, revenue by customer
- **Customer Report:** Customer acquisition, customer lifetime value
- **Product Report:** Best-selling products, inventory status

7. Pricing & Billing

Features:

- Multiple subscription plans (Free, Basic, Premium, Enterprise)
- Stripe payment integration
- Subscription management
- Usage tracking and limits
- Automatic billing with Stripe
- Webhook handling for payment events
- Plan upgrade/downgrade
- Cancel subscription
- Payment history

Subscription Plans:

- **Free:** Limited features
- **Basic:** Standard features
- **Premium:** Advanced features

- **Enterprise:** All features + priority support

8. AI Assistant

Technologies: OpenAI GPT-4

Features:

- Conversational AI assistant
- Context-aware responses
- Business decision support
- Auto-generate product descriptions
- Auto-generate customer profiles
- Invoice suggestions
- Report insights
- Natural language queries

Use Cases:

- "Create an invoice for customer X"
- "What were my top-selling products this month?"
- "Generate a product description for [product]"
- "Show me overdue invoices"

9. Multi-language Support

Languages: English, Arabic (RTL support)

Features:

- Dynamic language switching
- RTL (Right-to-Left) layout for Arabic
- Translation files for all UI text
- Language persistence in local storage

Implementation:

- i18next for internationalization
- React Context for language state
- Translation JSON files in `frontend/src/lang/`

10. Admin Panel

Features:

- User management
- View all users
- User activity monitoring
- Permission management
- System analytics
- Admin-only routes

- User role assignment

11. Dark Mode

Features:

- Toggle between light and dark themes
 - Persistent theme selection
 - Smooth theme transitions
 - CSS custom properties for theming
-

Frontend Architecture

State Management

Redux Toolkit is used for global state management:

```
// Store structure
{
  user: { ... },
  company: { ... },
  customers: { ... },
  products: { ... },
  invoices: { ... },
  reports: { ... },
  admin: { ... }
}
```

Data Fetching

TanStack Query (React Query) for server state management:

- Automatic caching
- Background refetching
- Optimistic updates
- Pagination support
- Error handling

```
const { data, isLoading, error } = useQuery({
  queryKey: ['customers'],
  queryFn: fetchCustomers
});
```

Routing

React Router v7 for client-side routing:

```

<Routes>
  { /* Public Routes */ }
  <Route path="/" element={<UnregisterHome />} />
  <Route path="login" element={<Login />} />
  <Route path="signup" element={<SignUp />} />
  <Route path="/auth/google/callback" element={<GoogleCallback />} />

  { /* Protected User Routes */ }
  <Route
    path="/dashboard"
    element={
      <PrivateRoute allowedRoles={["user"]} >
        <UserComponent />
      </PrivateRoute>
    }
  />

  <Route index element={<Dashboard />} />
  <Route path="invoice" element={<Invoices />} />
  <Route path="invoice/add" element={<AddInvoice />} />
  <Route path="invoice/edit/:id" element={<AddInvoice />} />
  <Route path="invoice/preview" element={<InvoicePreviewPage />} />
  <Route path="invoice/preview/:id" element={<InvoicePreviewPage />} />

  <Route path="product" element={<Products />} />
  <Route path="product/product/add" element={<AddProduct />} />
  <Route path="product/product/edit/:id" element={<EditProduct />} />
  <Route path="product/product/view/:id" element={<ViewProduct />} />

  <Route path="customer" element={<Customers />} />
  <Route path="customer/customer/add" element={<AddCustomer />} />
  <Route path="customer/customer/edit/:id" element={<EditCustomer />} />
  <Route path="customer/customer/view/:id" element={<ViewCustomer />} />

  <Route path="reports" element={<Reports />} />
  <Route
    path="pricing_billing"
    element={
      <Elements stripe={stripePromise}>
        <Pricing_billing />
      </Elements>
    }
  />
  <Route path="assistant" element={<AiAssistant />} />
  <Route path="settings" element={<Settings />} />
</Routes>

{ /* Protected Admin Routes */ }
<Route
  path="/admin"
  element={
    <PrivateRoute allowedRoles={["admin"]} >
      <AdminComponent />
    </PrivateRoute>
  }
/

```

```
    }  
  >  
    <Route path="mypath" element={<h1>This is for Admin</h1>} />  
  </Route>  
  
  { /* Logout & Error Routes */}  
  <Route path="logout" element={<UnregisterHome />} />  
  <Route path="*" element={<ErrorPage />} />  
</Routes>
```

Landing Page (UnregisterHome)

The **Landing Page** (`UnregisterHome.jsx`) is the first page visitors see when accessing PayFlow. It features a modern, animated, and fully responsive design with dark mode support.

Landing Page Sections

1. Hero Section

- Animated gradient background circles
- Main headline with fade-in animation
- Descriptive paragraph
- Focused feature cards with hover effects
- Dashboard preview image with floating animation
- Business owner testimonial card
- Call-to-action buttons: "Start Now" and "Start Demo"

2. Features Section

- Grid layout showcasing 6 main features:
 - Smart Dashboard
 - AI Invoices
 - Stock Management
 - Customer Database
 - AI Assistant
 - Business Reports
- Feature cards with hover animations (scale + rotate effects)
- Icon-based visual representation
- Detailed feature descriptions

3. How to Start Section

- Step-by-step guide with 3 cards:
 - Step 1: Add Products (with icon animation)
 - Step 2: Create Orders
 - Step 3: Generate Invoices
- Horizontal scroll for mobile devices
- AI Insights banner with rotating icon animation

4. Pricing Section

- Two pricing plans displayed:
 - **Free Starter Plan:** 0 EGP forever
 - Limited invoices & customers
 - Basic reporting
 - Email support
 - **Pay As You Go:** 2 EGP per invoice
 - Unlimited invoices & customers
 - Advanced AI features
 - Priority support
 - Custom reporting
- Animated pricing cards with hover effects
- Shine effect on hover
- Mobile-friendly horizontal scrolling

5. Testimonials Section

- Testimonial slider component
- Customer reviews and feedback
- Animated entrance effects
- Responsive design with shadow and border effects

6. Contact/Footer Section

- Company information
- Quick links
- Social media links
- Contact information
- Copyright notice

UI Components Used

- **HomeNavbar:** Landing page navigation bar with language switcher and dark mode toggle
- **FocusedCards:** Highlighted feature cards in hero section
- **FeatureCard:** Individual feature showcase cards
- **StepCard:** Step-by-step guide cards
- **TestimonialSlider:** Customer testimonials carousel
- **FloatingWhatsApp:** WhatsApp contact widget
- **Footer:** Page footer with links and information
- **Button:** Reusable button component with animations

Animations & Effects

- Framer Motion animations throughout:
 - **fadeInUp:** Elements fade in from bottom
 - **fadeInDown:** Elements fade in from top
 - **slideInLeft:** Elements slide in from left

- **slideInRight**: Elements slide in from right
- **scaleIn**: Elements scale up on appearance
- **floatingAnimation**: Continuous floating effect
- **staggerContainer**: Sequential animation of children
- Interactive hover effects:
 - Scale transformations
 - Rotation on hover
 - Shadow enhancements
 - Color transitions
- Back-to-top floating button:
 - Appears after scrolling 300px
 - Smooth scroll to top
 - Animated entrance/exit

Responsive Design

- Mobile-first approach
- Breakpoints for sm, md, lg, xl screens
- Horizontal scrolling sections on mobile
- Adaptive font sizes and spacing
- Touch-friendly interface

Component Structure

Atomic Design Pattern:

1. **Atoms**: Basic UI elements (buttons, inputs, icons)
2. **Molecules**: Simple component groups (form fields, cards)
3. **Organisms**: Complex components (forms, tables, modals)
4. **Templates**: Page layouts
5. **Pages**: Complete pages

Styling

Tailwind CSS with custom configuration:

- Utility-first CSS
- Custom theme colors
- Responsive design
- Dark mode support
- Custom components using **@layer**

Form Handling

React Hook Form for form management:


```
const { register, handleSubmit, formState: { errors } } = useForm();

const onSubmit = (data) => {
  // Submit logic
};
```

Notifications

React Hot Toast for user notifications:

```
toast.success('Invoice created successfully!');
toast.error('Failed to create invoice');
```

Backend Architecture

MVC Pattern

PayFlow follows a **modular MVC (Model-View-Controller)** pattern:

- **Models:** Database schemas (Mongoose models)
- **Controllers:** Business logic
- **Routes:** API endpoints

Middleware Stack

```
app.use(express.json());           // Parse JSON
app.use(cors({ origin: FRONTEND_URL })); // CORS
app.use(helmet());                 // Security headers
app.use(limiter);                  // Rate limiting
app.use(morgan('dev'));            // Logging
app.use(cookieParser());           // Parse cookies
app.use(passport.initialize());    // Passport auth
```

Error Handling

Centralized error handling:

```
// Custom error class
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.status = `${statusCode}`.startsWith('4') ? 'fail' : 'error';
    this.isOperational = true;
  }
}
```

```
    }  
  }  
  
  // Global error handler middleware  
  const globalErrorHandler = (err, req, res, next) => {  
    err.statusCode = err.statusCode || 500;  
    err.status = err.status || 'error';  
  
    res.status(err.statusCode).json({  
      status: err.status,  
      message: err.message  
    });  
  };  
};
```

Async Error Handling

Catch async wrapper:

```
const catchAsync = (fn) => {  
  return (req, res, next) => {  
    fn(req, res, next).catch(next);  
  };  
};  
  
// Usage  
exports.getCustomers = catchAsync(async (req, res, next) => {  
  const customers = await Customer.find();  
  res.json({ success: true, customers });  
});
```

Validation

Mongoose validation & custom validators:

```
const userSchema = new mongoose.Schema({  
  email: {  
    type: String,  
    required: [true, 'Email is required'],  
    unique: true,  
    lowercase: true,  
    validate: [validator.isEmail, 'Invalid email']  
  }  
});
```

File Uploads

Multer for file handling with dual storage support:

PayFlow supports two file storage strategies that automatically switch based on environment:

Local Storage (Development)

Used in local development environment:

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/company-logos/');
  },
  filename: (req, file, cb) => {
    const uniqueName = `${Date.now()}-${file.originalname}`;
    cb(null, uniqueName);
  }
});

const upload = multer({ storage });
```

Cloudinary Storage (Production/Vercel)

Used in production and Vercel deployments:

```
// Automatic detection in upload.config.js
const useCloudinary = process.env.VERCEL === '1' || process.env.USE_CLOUDINARY ===
'true';

if (useCloudinary) {
  console.log("🌩 Using Cloudinary for file storage (Vercel/Production)");
  // Uses cloudinary-multer-storage
} else {
  console.log("📁 Using local file storage (Development)");
  // Uses multer diskStorage
}
```

Supported Upload Types:

- Company logos (`uploadCompanyLogo`)
- Product images (`uploadProductImage`)

Static File Serving:

- Local files served from `/uploads` endpoint
- Cloudinary files served via CDN URLs

Scheduled Tasks

Node-Cron for automated tasks:

Cron jobs are enabled only in production mode or when explicitly enabled via environment variable.

```
// Cron jobs configuration in backend/src/index.js
const isProduction = process.env.NODE_ENV === 'production';

if (isProduction || process.env.ENABLE_CRON === 'true') {
  // Daily billing at midnight
  cron.schedule('0 0 * * *', async () => {
    console.log('Starting scheduled batch charge...');
    await chargeAllUsers();
    console.log('Finished scheduled batch charge');
  });
  console.log('✅ Cron job enabled - Daily billing charges scheduled');
} else {
  console.log('⚠️ Cron jobs disabled in development mode');
}
```

Cron Job Features:

- Automatic daily billing charge at midnight
- Production/development mode detection
- Manual override with `ENABLE_CRON=true` environment variable
- Logs for tracking cron job execution

API Rate Limiting

Express Rate Limit:

```
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // 100 requests per window
  message: 'Too many requests, please try again'
});
```

Caching Strategy

Redis for caching:

- Cache frequently accessed data
- Session management
- Temporary data storage
- Rate limit tracking

Security & Authentication

Security Measures

1. **Helmet.js** - Sets security HTTP headers
2. **CORS** - Configured for specific origin

3. **Rate Limiting** - Prevents brute-force attacks
4. **Password Hashing** - Bcrypt with salt rounds
5. **JWT** - Secure token-based authentication
6. **Input Validation** - Mongoose validators
7. **SQL Injection Protection** - Mongoose ODM
8. **XSS Protection** - Input sanitization
9. **HTTPS** - Required in production

Authentication Flow

1. User registers/logs in with credentials
2. Server validates credentials
3. Server generates JWT token
4. Token sent to client (cookie + response)
5. Client includes token in subsequent requests
6. Server validates token via middleware
7. Request processed if token valid

JWT Structure

```
{
  payload: {
    id: user._id,
    email: user.email,
    role: user.role
  },
  expiresIn: '7d'
}
```

Protected Routes

```
const protect = catchAsync(async (req, res, next) => {
  let token;

  // Get token from cookie or header
  if (req.cookies.token) {
    token = req.cookies.token;
  } else if (req.headers.authorization?.startsWith('Bearer ')) {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return next(new AppError('Not authorized', 401));
  }

  // Verify token
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

```
// Check if user still exists
const user = await User.findById(decoded.id);
if (!user) {
  return next(new AppError('User no longer exists', 401));
}

req.user = user;
next();
});
```

Role-Based Access Control

```
const restrictTo = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return next(new AppError('Not authorized', 403));
    }
    next();
  };
};

// Usage
router.get('/admin', protect, restrictTo('admin'), getAdminData);
```

Deployment

Vercel Deployment

PayFlow is configured for deployment on **Vercel**.

Backend Deployment

backend/vercel.json configuration:

```
{
  "version": 2,
  "builds": [
    {
      "src": "src/index.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "src/index.js"
    }
  ]
}
```

```
]
}
```

Deployment steps:

1. Push code to GitHub
2. Connect GitHub repo to Vercel
3. Configure environment variables in Vercel dashboard
4. Deploy backend

Frontend Deployment

1. Build frontend: `npm run build`
2. Deploy to Vercel
3. Configure environment variables
4. Set build command: `vite build`
5. Set output directory: `dist`

Environment Variables (Production)

Backend:

- `NODE_ENV=production`
- `MONGO_URL` - MongoDB Atlas connection string
- `JWT_SECRET` - Strong secret key
- `STRIPE_SECRET_KEY` - Production Stripe key
- `OPENAI_API_KEY` - OpenAI API key
- `FRONTEND_URL` - Production frontend URL
- `REDIS_URL` - Production Redis URL

Frontend:

- `VITE_API_URL` - Production API URL
- `VITE_STRIPE_PUBLISHABLE_KEY` - Production Stripe public key

Database Hosting

MongoDB Atlas:

1. Create cluster on MongoDB Atlas
2. Configure IP whitelist (0.0.0.0/0 for Vercel)
3. Create database user
4. Get connection string
5. Add to environment variables

Monitoring & Logging

- **Morgan** - HTTP request logging
- **MongoDB Logs** - Database query logs

- **Vercel Analytics** - Performance monitoring
 - **Error Tracking** - Centralized error handler
-

Testing

Backend Testing

Test structure (to be implemented):

```
// Example test
describe('User Authentication', () => {
  it('should register a new user', async () => {
    const res = await request(app)
      .post('/api/auth/register')
      .send({
        firstName: 'John',
        lastName: 'Doe',
        email: 'john@test.com',
        password: 'password123'
      });

    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('token');
  });
});
```

Frontend Testing

Test structure (to be implemented):

```
// Component test
test('renders login form', () => {
  render(<Login />);
  expect(screen.getByLabelText(/email/i)).toBeInTheDocument();
  expect(screen.getByLabelText(/password/i)).toBeInTheDocument();
});
```

Manual Testing Checklist

- ☐ User registration and login
- ☐ Company creation and management
- ☐ Customer CRUD operations
- ☐ Product CRUD operations
- ☐ Invoice creation and PDF generation
- ☐ Report generation (all types)
- ☐ Payment processing with Stripe
- ☐ AI assistant functionality

- ☐ Dark mode toggle
 - ☐ Language switching
 - ☐ Admin panel access control
 - ☐ Responsive design on mobile
 - ☐ Error handling and validation
-

Contributing Guidelines

Code Style

- Use **ES6+** syntax
- Follow **Airbnb JavaScript Style Guide**
- Use **meaningful variable names**
- Add **comments** for complex logic
- Keep functions **small and focused**

Git Workflow

1. Create a new branch: `git checkout -b feature/your-feature`
2. Make changes and commit: `git commit -m "Add feature"`
3. Push to branch: `git push origin feature/your-feature`
4. Create Pull Request
5. Wait for code review
6. Merge after approval

Commit Message Format

```
type(scope): subject  
  
body (optional)  
  
footer (optional)
```

Types:

- **feat** - New feature
- **fix** - Bug fix
- **docs** - Documentation
- **style** - Formatting
- **refactor** - Code restructuring
- **test** - Tests
- **chore** - Maintenance

Example:

```
feat(invoice): add PDF export functionality
```

- Added PDFKit integration
- Created PDF template
- Added download button

Closes #123

Pull Request Guidelines

1. Update documentation if needed
2. Add tests for new features
3. Ensure all tests pass
4. Follow code style guidelines
5. Provide clear PR description
6. Link related issues

Troubleshooting

Common Issues & Solutions

1. MongoDB Connection Error

Error: `MongooseServerSelectionError: connect ECONNREFUSED`

Solution:

- Check if MongoDB is running: `mongod`
- Verify `MONGO_URL` in `.env`
- Check network connectivity
- Ensure correct database name

2. JWT Token Invalid

Error: `JsonWebTokenError: invalid token`

Solution:

- Clear browser cookies
- Check `JWT_SECRET` is set in `.env`
- Ensure token format: `Bearer <token>`
- Re-login to get new token

3. Stripe Payment Fails

Error: `Stripe API error`

Solution:

- Verify `STRIPE_SECRET_KEY` is correct
- Use test cards in development
- Check Stripe dashboard for errors
- Ensure webhook is configured

4. CORS Error

Error: `CORS policy: No 'Access-Control-Allow-Origin' header`

Solution:

- Check `FRONTEND_URL` in backend `.env`
- Verify CORS configuration in `index.js`
- Ensure credentials: true in both frontend and backend

5. File Upload Error

Error: `Multer error: File too large`

Solution:

- Check file size limits in multer config
- Increase limits if needed
- Verify file type is allowed
- Check disk space

6. OpenAI API Error

Error: `OpenAI API rate limit exceeded`

Solution:

- Check API key is valid
- Monitor API usage
- Implement request throttling
- Upgrade OpenAI plan if needed

7. Port Already in Use

Error: `EADDRINUSE: address already in use :::5000`

Solution:

```
# Windows
netstat -ano | findstr :5000
taskkill /PID <PID> /F

# Linux/Mac
lsof -ti:5000 | xargs kill -9
```

8. Module Not Found

Error: `Cannot find module 'module-name'`

Solution:

```
# Clear node_modules and reinstall
rm -rf node_modules package-lock.json
npm install
```

9. Build Errors

Error: `Build failed`

Solution:

- Clear cache: `npm cache clean --force`
- Delete `node_modules` and reinstall
- Check for syntax errors
- Update dependencies: `npm update`

10. Database Seeding Fails

Error: `Seeding failed`

Solution:

- Ensure MongoDB is running
- Drop existing database: `npm run delete`
- Re-run seed: `npm run seed`
- Check seed data format

Performance Optimization

Backend Optimization

1. Database Indexing

```
userSchema.index({ email: 1 });
invoiceSchema.index({ customer: 1, createdAt: -1 });
```

2. Query Optimization

- Use `.select()` for specific fields
- Use `.lean()` for read-only queries
- Implement pagination

3. Caching with Redis

- Cache frequently accessed data
- Set appropriate TTL
- Invalidate cache on updates

4. Async Operations

- Use `Promise.all()` for parallel operations
- Implement job queues for heavy tasks

Frontend Optimization

1. Code Splitting

```
const Dashboard = lazy(() => import('./components/Dashboard'));
```

2. Image Optimization

- Compress images
- Use WebP format
- Lazy load images

3. Bundle Size

- Analyze bundle: `npm run build -- --analyze`
- Remove unused dependencies
- Use dynamic imports

4. React Query Caching

- Set appropriate stale time
- Use query invalidation
- Implement optimistic updates

API Response Format

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": { ... },
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 100,
    "pages": 10
  }
}
```

Error Response

```
{
  "success": false,
  "status": "fail",
  "message": "Error message here",
  "errors": [ ... ]
}
```

License

This project is proprietary software developed for educational purposes as part of the DEPI Course.

Contact & Support

For questions, issues, or contributions:

- **GitHub:** [SerajEldeen/PayFlow](#)
 - **Email:** support@payflow.com
 - **Documentation:** This file
-

Acknowledgments

- **DEPI Course** - Educational program
 - **Team Members** - For their dedication and hard work
 - **Open Source Community** - For the amazing tools and libraries
-

Changelog

Version 1.0.0 (Current)

- Initial release
 - Complete CRM and billing features
 - AI assistant integration
 - Multi-language support
 - Stripe payment integration
 - Admin panel
 - Dark mode
-

Roadmap

Upcoming Features

- ☐ Email automation
- ☐ SMS notifications
- ☐ Mobile app (React Native)

- ☐ Advanced analytics dashboard
- ☐ Inventory management
- ☐ Multi-currency support
- ☐ Team collaboration features
- ☐ API documentation (Swagger)
- ☐ Automated testing suite
- ☐ Performance monitoring
- ☐ Customer portal
- ☐ Recurring invoices
- ☐ Payment reminders
- ☐ Document storage
- ☐ Integration marketplace

Appendix

Useful Commands

```
# Development
npm run dev           # Run both frontend and backend
npm run seed          # Seed database
npm run delete        # Delete all data

# Backend
cd backend
npm start             # Start production server
npm run dev           # Start development server

# Frontend
cd frontend
npm run dev           # Start development server
npm run build          # Build for production
npm run preview        # Preview production build
npm run lint          # Run ESLint

# Git
git status            # Check status
git add .             # Stage all changes
git commit -m "message" # Commit changes
git push origin dev   # Push to dev branch
git pull origin dev   # Pull from dev branch
```

Project Statistics

- **Total Files:** 100+
- **Lines of Code:** 15,000+
- **Dependencies:** 60+
- **API Endpoints:** 50+
- **Database Models:** 7

- **React Components:** 40+
-

Last Updated: December 2, 2025

Version: 1.0.0

Maintained By: PayFlow Development Team

Documentation Notes:

- This documentation has been updated to reflect the current state of the codebase
- All configurations, file structures, and API endpoints are verified against the implementation
- Backend uses port 3000 (not 5000)
- File uploads support both local storage (development) and Cloudinary (production)
- Landing page features full animation suite with Framer Motion
- Cron jobs are disabled in development mode by default